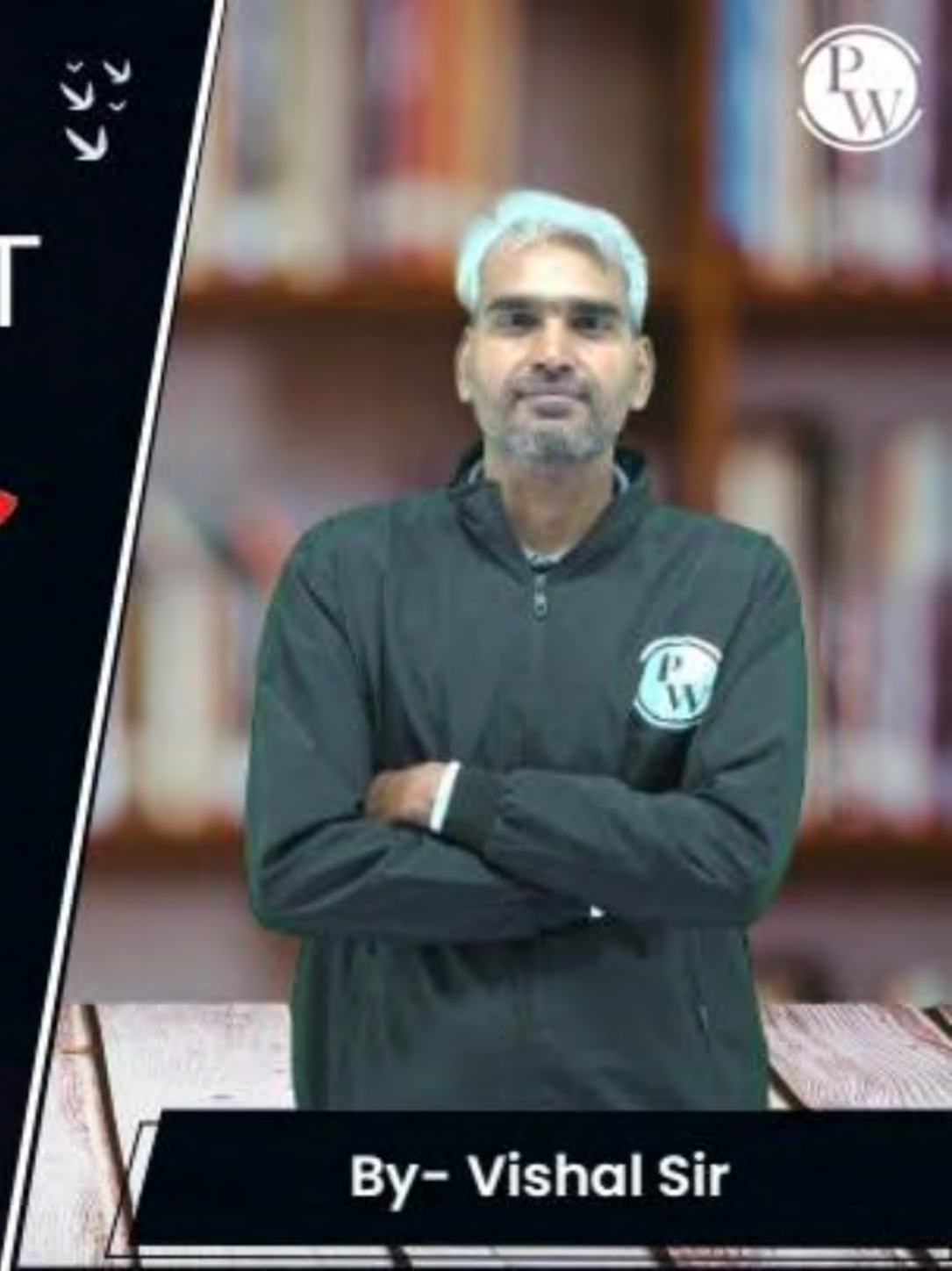
Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 08



By- Vishal Sir

Recap of Previous Lecture



Topic LL(1) Parsing Table construction

Topic LL(1) Parsing Algorithm

Slide

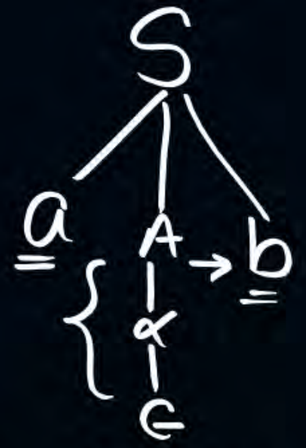
Topics to be Covered



- Topic: Parsing using LL(1) parsing table
- Topic: Conditions to check LL(1) grammar
- Topic: Introduction to Bottom-up Parsers
- Topic: Augmented Grammar
- Topic: LR(0) items



Topic : Correction



Construction of LL(1) parsing table.

for each production $A \rightarrow \alpha$ of the grammar repeat the following.

① Add $A \rightarrow \alpha$ under $M[A, b]$, $\forall b \in first(\alpha)$

② If $\epsilon \in first(\alpha)$, then add $A \rightarrow \alpha$ under $M[A, c]$, $\forall c \in follow(A)$

Slide



Topic : LL(1) Parsing Algorithm

Initially PUSH $ into the stack and then PUSH starting symbol of the grammar into the stack.

Let X is the top of the stack symbol and 'a' be the look-ahead symbol pointed by input pointer, and let M is the LL(1) parsing table, then:

1. If X = a = $ then parsing is is successful
2. If (X = a) ≠ $ then POP X from stack and increment input pointer.
3. If X is a non-terminal and M[X, a] contain the production X→uvw then replace X by uvw in the reverse order {i.e., u should appear at the top of stack}.
4. If X is a non-terminal and M[X, a] is a blank entry in the table, then indicate syntax error.
5. If X is a terminal and X ≠ a, then indicate syntax error.

Slide



Topic : Parsing using LL(1) Parsing Algorithm

H.W.

Consider the following grammar

$E \rightarrow TE'$

$E' \rightarrow +TE'/\varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'/\varepsilon$

$F \rightarrow id/(E)$

M	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Try parsing the input string $id + id * id\$$ using LL(1) parser.

Slide

$E \rightarrow TE'$

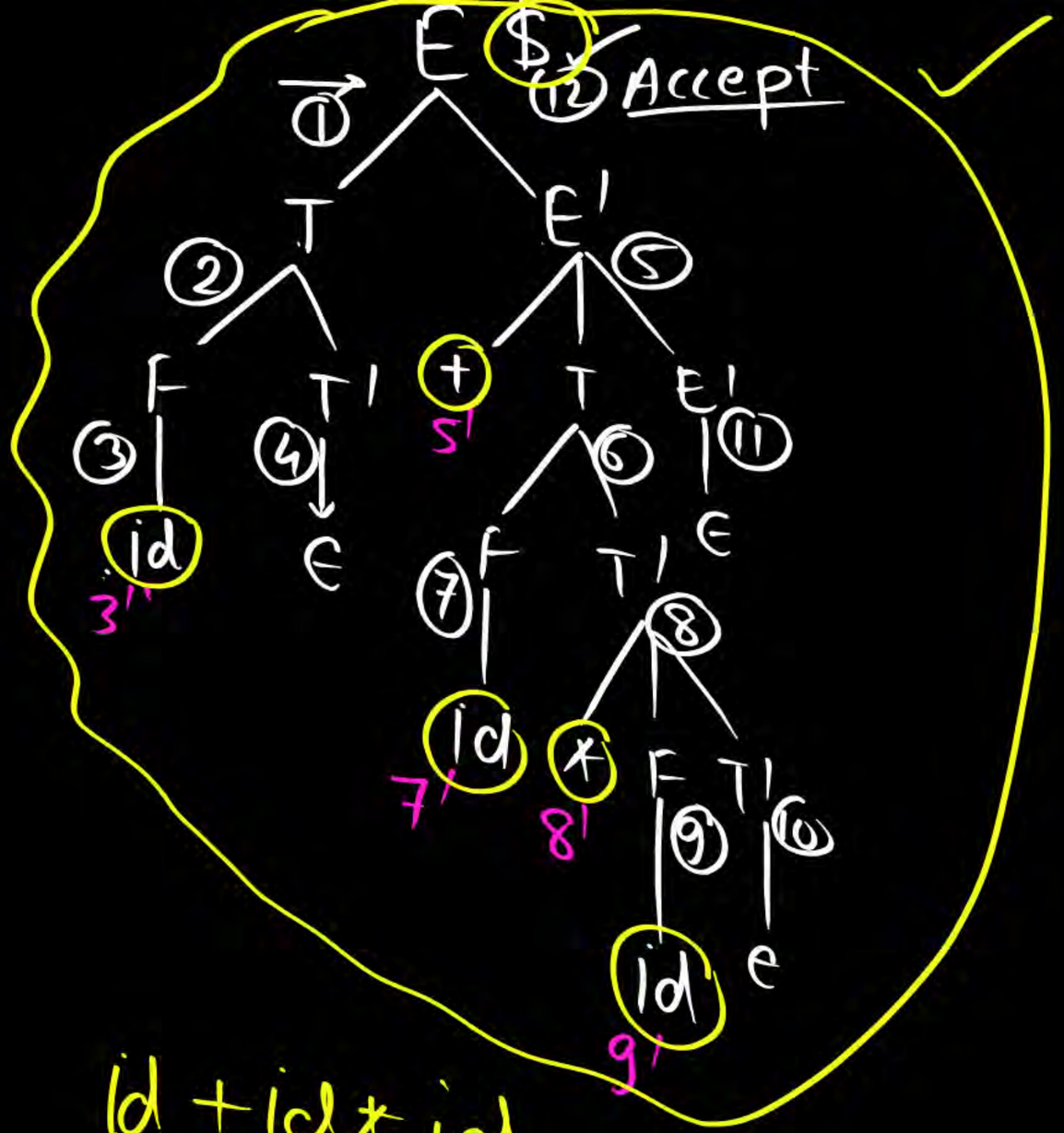
$E' \rightarrow +TE' / \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \epsilon$

$F \rightarrow id$

$F \rightarrow (E)$



id + id * id

Topic : Parsing using LL(1) Parsing Algorithm



Stack

input

Stack		input
$ E	$M[E,id] = E \rightarrow TE'$$ then Replace E by TE'	id + id * id $
$ E' T	$M[T,id] = T \rightarrow FT'$	id + id * id $
$ E' T' F	$M[F,id] = F \rightarrow id$	id + id * id $
$ E' T' id	(id == id) ≠ $ Pop, Top of stack and increment i/p ptr.	id + id * id $

Top

look-ahead

Slide

id popped

$	E'	T'

$M[T', +] = T' \rightarrow \epsilon$

i.e. Replace T' by ϵ

i.e. Replace T' by nothing

i.e. Simply pop T'

+ id * id $

↑ i/p pointer incremented

$	E'

$M[E', +] = E' \rightarrow +TE'$

+ id * id $

↑

E' Replaced by +TE'

$	E'	T	+

$(+ == +) \neq S$

∴ Pop & increment

+ id * id $

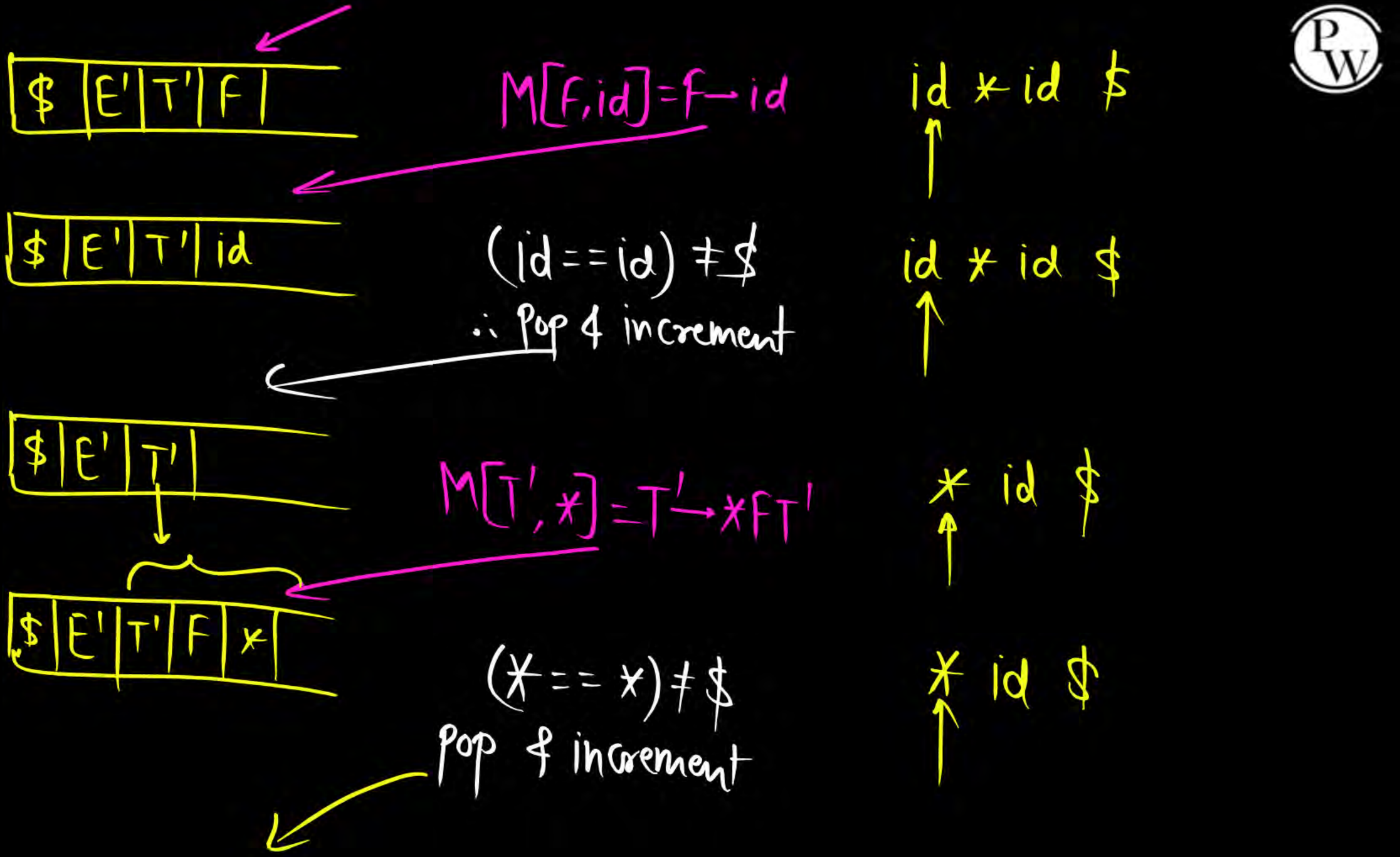
↑

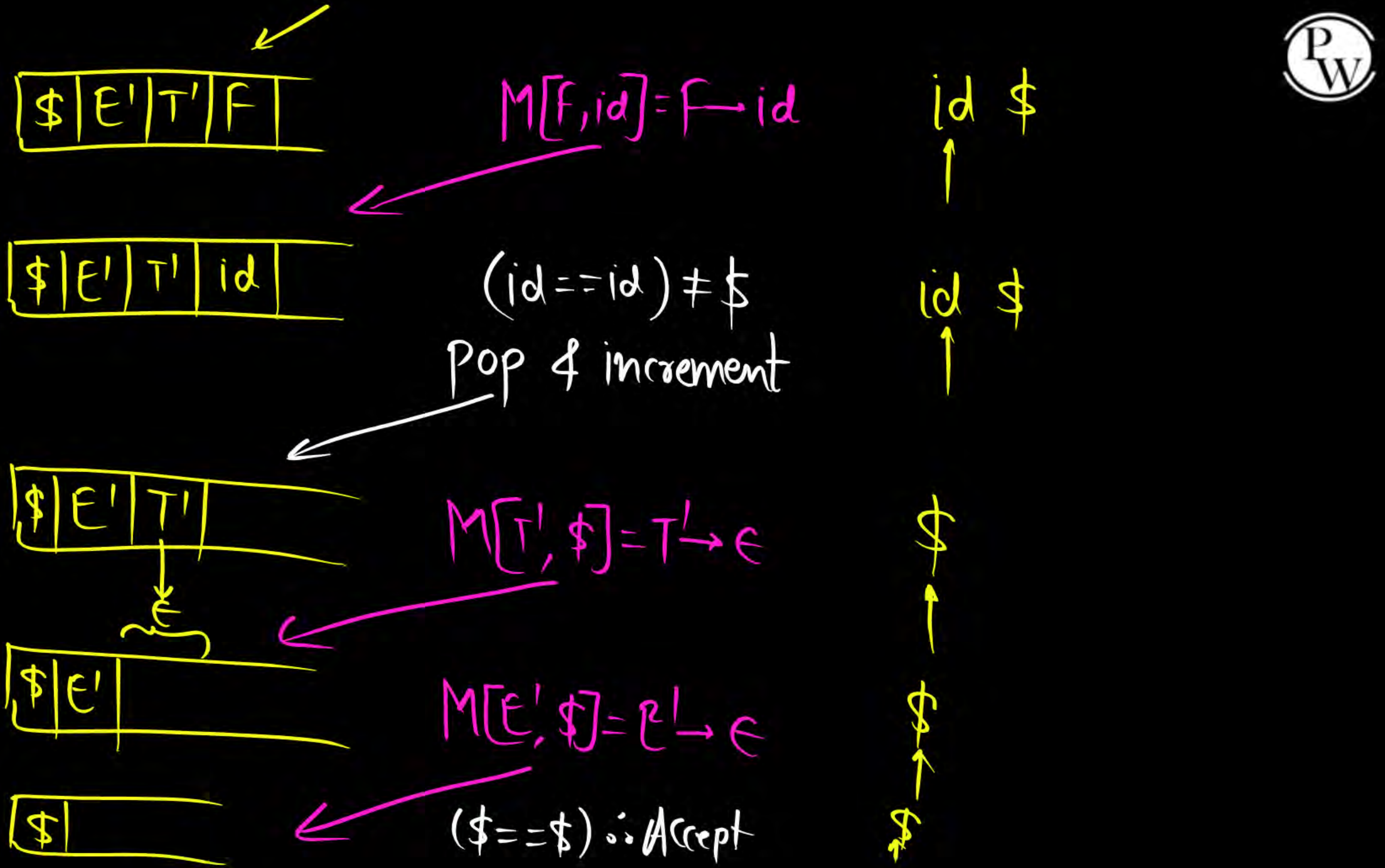
$	E'	T

$M[T, id] = T \rightarrow FT'$

id * id $

↑


$ E' T' F
M[F,id]=F→id
id * id $
$ E' T' id
(id==id)≠$
∴ Pop & increment
id * id $
$ E' T'
M[T',*]=T'→*FT'
* id $
$ E' T' F *
(*==*)≠$
Pop & increment
* id $


$ E' T' F
M[F, id] = F → id
id $
$ E' T' id
(id == id) ≠ $
Pop & increment
id $
$ E' T'
M[T', $] = T' → ϵ
$
$ E'
M[E', $] = E' → ϵ
$
$
$
($ == $) ∴ Accept



Topic : Conditions to check LL(1) grammar



1. Left Recursive grammar Can not be LL(1)

2. Grammar With Common-prefix Can not be LL(1)

3. If there is any non-terminal in the grammar Corresponding to which there is only one production in the grammar, then that production will not create any problem

Slide

(4) If $A \rightarrow \alpha_1 / \alpha_2 / \alpha_3 \ldots$ are the productions in the grammar then grammar can be LL(1) only if $first(\alpha_1)$, $first(\alpha_2)$ and $first(\alpha_3)$ are disjoint, i.e.,

(i) $first(\alpha_1) \cap first(\alpha_2) = \phi$

(ii) $first(\alpha_2) \cap first(\alpha_3) = \phi$

(iii) $first(\alpha_1) \cap first(\alpha_3) = \phi$



Topic : Conditions to check LL(1) grammar

(5) Let $A \rightarrow \alpha_1 / \alpha_2 / \in$ are the productions in the grammar,

Grammar can be LL(1) only if first(α_1), first(α_2) and follow(A) are disjoint.

i.e., (i) first(α_1) $\cap$ first(α_2) = ϕ

(ii) first(α_1) $\cap$ follow(A) = ϕ

(iii) first(α_2) $\cap$ follow(A) = ϕ

Slide



Topic : Conditions to check LL(1) grammar

Check whether the following grammar is LL(1) or not ?

Ans

LL(1)

Yes

No

1)

LL(1)

- S→AB } Single production of S, ∴ No problem
- A→a } — " — — A, — "
- B→b } — " — — B, — "

No problem because of any production

∴ LL(1) gramma

Slide



Topic : Conditions to check LL(1) grammar

LL(1)
Yes
NO

Check whether the following grammar is LL(1) or not ?

2)
- S→aABe
- A→Abc/b
- B→d

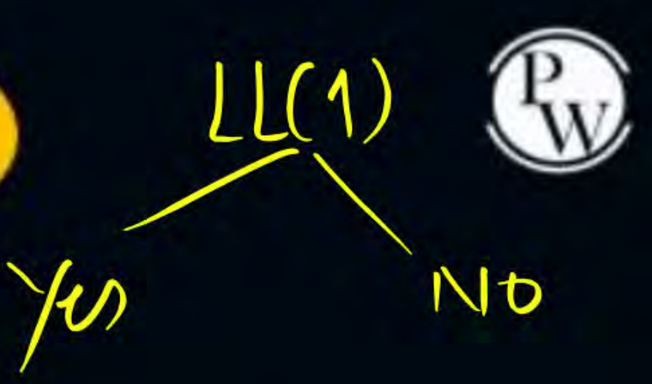
Left Recursion ∴ grammar can not be LL(1)

Not LL(1)

Slide



Topic : Conditions to check LL(1) grammar



Check whether the following grammar is LL(1) or not ?

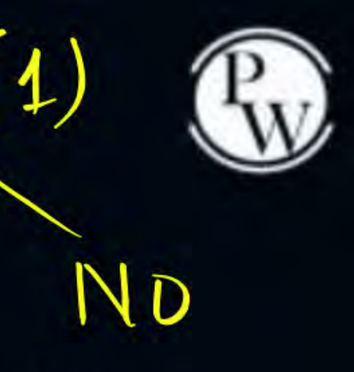
3) LL(1)

- $S \rightarrow AB$ ✓ Single production, ∴ No problem
- $A \rightarrow a/b$
  - α_1, $first(\alpha_1) = first(a) = \{a\}$
  - α_2, $first(\alpha_2) = first(b) = \{b\}$
  - $\{a\} \cap \{b\} = \phi$, ∴ No problem
- $B \rightarrow b/c$
  - α_2, $first(\alpha_2) = first(c) = \{c\}$
  - α_1, $first(\alpha_1) = first(b) = \{b\}$
  - $\{c\} \cap \{b\} = \phi$, ∴ No problem

Slide



Topic : Conditions to check LL(1) grammar



Check whether the following grammar is LL(1) or not ?

4) S → aSA / ϵ

first(aSA) = {a}

follow(S) = {$, c}

{a} ∩ {$, c} = ∅, ∴ No problem

A → c / ϵ

follow(A) = follow(S) = {$, c}

first(c) = {c}

{c} ∩ {$, c} = {c} ≠ ∅

Not equal to ∅

∴ Not LL(1)

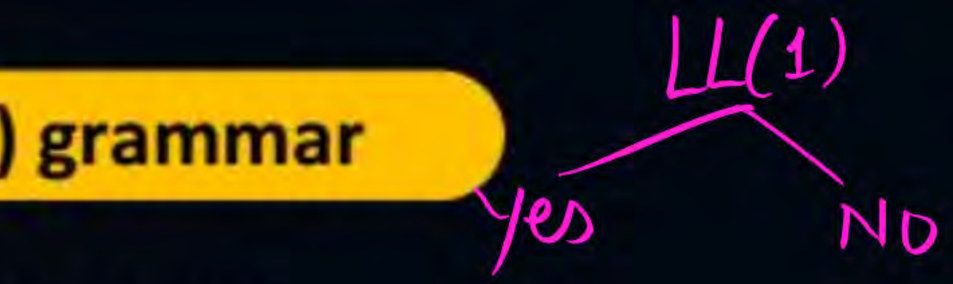
Not LL(1)

		c
A		A → c A → ϵ

Slide



Topic : Conditions to check LL(1) grammar



Check whether the following grammar is LL(1) or not ?

5) LL(1)

S → 1ABb / ϵ
first(1ABb) = {1}
follow(S) = {$}
{1} ∩ {$} = ∅ ∴ No problem

A → 1Ab / bC
{1} ∩ {b} = ∅ ∴ No problem

B → aB / b
{a} ∩ {b} = ∅ ∴ No Problem

C → a
Single production ∴ No problem

Slide



Topic : Conditions to check LL(1) grammar

LL(1)
Yes
NO

Check whether the following grammar is LL(1) or not ?

α_1, first(α_1) = first(aAbB) = {a}

α_2, first(α_2) = first(bAaB) = {b}

6) S → aAbB / bAaB / ϵ

ϵ, ∴ follow(S) = {$, a, b}

{a} ∩ {$, a, b} ≠ ∅

∴ Not LL(1)

Not LL(1)

A → S } No problem

B → S } No problem

Slide

LL(1) parsing Complete



Topic : Bottom-up Parsers

Because of Shift & Reduce actions, Bottomup Parsers are also known as Shift-Reduce Parser

- Bottom Up Parser starts constructing the parse tree, starting from the given input string and proceeds until the starting symbol of the grammar is obtained.
  (i.e. Perform a series of reductions)
- If a parser uses bottom up approach for parsing the string, then it follows reverse of rightmost derivation in the construction of parse tree.

Slide



Topic : Types of Bottom-up Parsers

Bottom-up Parser

L R parser

Can work Only with Unambiguous grammars

(Reverse of Rightmost Derivation)

Left to Right Scan

Operator Precedence Parser

it Can work With Unambiguous as well as Ambiguous grammar

But grammar must be operator grammar

Slide



Topic : Types of LR Parsers



LR Parsers

- LR(0) Parser
- SLR(1) Parser
  - Simple LR Parser
- CLR(1) Parser
  - Canonical LR Parser
    - Also known as LR(1) Parser
- LALR(1) Parser
  - LookAhead LR Parser

Slide



Topic : Power of LR Parsers

Power of LR Parser

$$CLR(1) \geqq LALR(1) \geqq SLR(1) \geqq LR(0)$$

- CLR(1) is most powerful LR Parser
- LR(0) is least powerful LR Parser



LL(1) grammar $\subset$ CLR(1) grammar

Bottom-up parser are much more powerful compared to Top-down Parser, but design of bottom-up parser is more Complex

Slide



2 mins Summary

Topic LL(1) Parser

Topic Different types of bottom up parsers

Slide

THANK - YOU

Slide